

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 51 01-1-ЛУ

Исполнители:

Студент группы БДРИП241

_____ / С. В. Гаврилин /
«__» _____ 2026 г.

Студент группы БДРИП242

_____ / А. А. Клушин /
«__» _____ 2026 г.

Москва 2026

Инва.№ подп	Подп. и дата	Взам. инв.№	Инва.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.09.03-04 51 01-1-ЛЮ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Программа и методика испытаний

RU.17701729.09.03-04 51 01-1

Листов 23

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

Москва 2026

АННОТАЦИЯ

Настоящий документ является программой и методикой испытаний (ГОСТ 19.301-79) серверной и клиентской частей программы «Платформа для no-code автоматизации бизнес-процессов». Документ содержит описание объекта испытаний, цели проводимых испытаний, перечень технических и программных средств, описание автоматических тестов и методики ручной проверки функций системы.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.105-78 [1]: Общие требования к программным документам.
2. ГОСТ 19.106-78 [2]: Требования к программным документам, выполненным печатным способом.
3. ГОСТ 19.301-79 [3]: Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения оформляются согласно ГОСТ 19.603-78, ГОСТ 19.604-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	5
1.1. Наименование программы	5
1.2. Краткая характеристика	5
2. ЦЕЛЬ ИСПЫТАНИЙ	6
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	7
3.1. Функциональные характеристики	7
3.2. Требования к надёжности	7
3.3. Требования к производительности	7
4. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	8
4.1. Технические средства	8
4.2. Программные средства	8
4.3. Порядок проведения испытаний	8
5. АВТОМАТИЧЕСКИЕ ТЕСТЫ	9
5.1. Состав автоматических тестов	9
5.2. Запуск автоматических тестов CI	9
6. МЕТОДЫ ИСПЫТАНИЙ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	11
6.1. Испытание создания и редактирования workflow	11
6.2. Испытание системы триггеров	11
6.3. Испытание HTTP-узла	12
6.4. Испытание потоков данных	13
6.5. Испытание Code Node	13
6.6. Испытание параллельного исполнения и отказоустойчивости	14
6.7. Испытание интерфейса просмотра состояния и мониторинга	14
6.8. Испытание версионирования	15
6.9. Испытание content-addressed-хранения в MinIO	15
6.10. Испытание автодокументации REST API	16
6.11. Испытание адаптивной вёрстки клиентской части	16
6.12. Испытание надёжности	16
6.13. Испытание режима пошаговой отладки workflow	17
7. ИСПЫТАНИЕ НЕФУНКЦИОНАЛЬНЫХ ХАРАКТЕРИСТИК	18

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

7.1. Производительность REST API	18
7.2. WebSocket-задержка	18
7.3. Покрытие тестами	18
8. СВОДНЫЕ ПОКАЗАТЕЛИ	19
9. ПРИЛОЖЕНИЕ. ТРАССИРОВКА ТРЕБОВАНИЙ ОБЩЕГО ТЗ К РАЗДЕЛАМ ПМИ	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

«Платформа для no-code автоматизации бизнес-процессов» — командный курсовой проект, выполненный в составе двух участников (Гаврилин С.В. — клиентская часть; Клушин А.А. — серверная часть, инфраструктура, автоматизация тестирования).

1.2. Краткая характеристика

Объект испытаний — полный программный комплекс из двух взаимодействующих частей. Серверная часть — Kotlin 2.2.21 на Spring Boot 3.5.3 (JDK 24), включающая движок исполнения workflow, исполнители узлов десяти типов (включая BranchSplit/BranchMerge для A/B-экспериментов), Docker-песочницу пользовательского Python- и JavaScript-кода, контент-адресуемое хранилище MinIO с дедупликацией, двухуровневое версионирование с именованными снапшотами, систему триггеров, WebSocket-брокер, серверный модуль продуктовой A/B-аналитики со статистическим тестированием, автодокументацию REST API. Клиентская часть — одностраничное приложение на Angular 19.2 (TypeScript 5.6), включающее визуальный редактор workflow с холстом drag-and-drop, инспектором конфигурации узлов, панелью запусков и панелью подробного просмотра одного запуска, панелью именованных снапшотов, вкладкой продуктовой A/B-аналитики, онбординг-туром.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Проверка корректности выполнения программой четырнадцати функциональных требований раздела 4.1.1 общего технического задания (RU.17701729.09.03-04 ТЗ 01-1) и достижения целевых нефункциональных показателей, заявленных в общей пояснительной записке (RU.17701729.09.03-04 81 01-1). Проверка целостности и согласованности взаимодействия серверной и клиентской частей в полном end-to-end-сценарии.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Функциональные характеристики

К проверке подлежат четырнадцать функциональных требований, перечисленных в разделе 4.1.1 общего ТЗ: визуальный редактор workflow; система триггеров (manual, scheduler, webhook); HTTP-узел; узлы потоков данных (filter, map, reduce, foreach, flatmap); Code Node с Python- и JavaScript-песочницами; параллельное исполнение независимых ветвей с обработкой сбоев; интерфейс просмотра состояния; feature-флаги; WebSocket-трансляция событий в реальном времени; content-addressed-хранение конфигов в MinIO с дедупликацией; двухуровневое версионирование (ревизии + именованные snapshot-ы); A/B-аналитика workflow (BranchSplit/Merge узлы + сервер аналитики с z-тестом); снапшоты workflow уровня документа; режим пошаговой отладки workflow с возможностью запуска одной выбранной ноды.

3.2. Требования к надёжности

Приложение не должно аварийно завершаться при некорректных входных данных. Code Node должен выполняться исключительно в Docker-песочнице с полной изоляцией. Ошибка одного узла workflow не должна приводить к потере данных запуска: статус каждого узла, входы и выходы должны сохраняться в БД. Восстановление к ранее сохранённой версии workflow должно создавать новую ревизию (append-only), а не модифицировать существующие.

3.3. Требования к производительности

95-й перцентиль REST API на чтение — не более 500 мс при типовой нагрузке. Время от приёма триггера до перехода первого узла в running — не более 5 секунд. 95-й перцентиль WebSocket-задержки в локальной сети — не более 200 мс. Время отклика веб-интерфейса — не более 2 секунд.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

4.1. Технические средства

Один компьютер: процессор не менее 4 ядер x86_64; ОЗУ не менее 8 ГБ (рекомендуется 16 ГБ); диск не менее 20 ГБ; стабильное подключение к сети для загрузки Docker-образов.

4.2. Программные средства

Операционная система: Linux (Ubuntu 22.04 LTS и совместимые), macOS 12+, Windows 10/11 с WSL2. Браузер: Chrome 90+, Firefox 90+, Safari 15+, Edge 90+. Сервер: JDK 24, Apache Maven 3.9+, Docker Engine 20.10+, Docker Compose, PostgreSQL 16, MinIO. Клиент: Node.js 20 LTS, npm 10+. Дополнительно: curl/HTTPie/Postman для REST-запросов, утилита psql для прямых SQL-запросов.

4.3. Порядок проведения испытаний

Испытания проводятся в три этапа.

Этап 1: запуск всех автоматических тестов (раздел 5). Команды: `cd backend && mvn verify; cd frontend && npm ci && npm test -- --watch=false --browsers=ChromeHeadless; cd frontend && npx playwright test`. Время прогона — не более 12 минут. Критерий: 100% тестов завершились с результатом success.

Этап 2: ручная проверка функциональных характеристик по методикам раздела 6. Каждая методика выполняется последовательно с фиксацией фактических результатов.

Этап 3: проверка надёжности и нефункциональных характеристик (раздел 7).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. АВТОМАТИЧЕСКИЕ ТЕСТЫ

5.1. Состав автоматических тестов

Сводные объёмы тестового покрытия представлены в таблице ниже. Состав включает: набор автоматических тестов серверной части — модульные тесты пяти операций потоков в `DataflowNodeExecutorsTest`, интеграционные тесты сквозных сценариев на `Testcontainers` в `MvpIntegrationTests`, тесты загрузки Spring-контекста в `AllaApplicationTests`, модульные тесты узлов разделения и слияния ветвей (`BranchSplitNodeExecutorTest`, `BranchSplitStrategiesTest`, `SplitEnvelopeTest`, `BranchMergeNodeExecutorTest`), интеграционные тесты исполнения с `BranchSplit/Merge` (`WorkflowExecutionServiceBranchTest`), тесты модуля А/В-аналитики (`AbAnalyticsServiceTest`, `AbAnalyticsControllerTest`, `StatTestTest`), тесты модели соединений графа (`ConnectionSkeletonTest`); модульные тесты клиентской части для апи-фасадов и маперов в файлах `run.api.spec.ts`, `trigger.api.spec.ts`, `workflow.api.spec.ts`, `workflow.facade.spec.ts`, `workflow.mapper.spec.ts`, для сервисов уровня приложения — `workflow.service.spec.ts`, `execution.service.spec.ts`, `workflow-validator.service.spec.ts`, для WebSocket-клиента — `workflow-ws.service.spec.ts`, для компонента аналитики — `analytics-panel.component.spec.ts`; end-to-end-тесты в файлах Playwright `api.spec.ts`, `canvas-editing.spec.ts`, `intro-and-guide.spec.ts`, `modals.spec.ts`, `runs.spec.ts`, `triggers.spec.ts`, `versions.spec.ts`, `workflows.spec.ts`.

Категория	Технология	Количество	Команда запуска
Серверные модульные	JUnit 5	около 55	<code>mvn test</code>
Серверные интеграционные	Testcontainers	около 12	<code>mvn verify</code>
Клиентские модульные	Jasmine + Karma	около 65	<code>npm test -- --watch=false</code>
End-to-end	Playwright	около 95	<code>npx playwright test</code>
Покрытие backend	JaCoCo	не менее 90 %	<code>mvn verify → target/site/jacoco/</code>
Итого тестов	—	около 230	—

Таблица 1. Сводные объёмы автоматических тестов

Все автоматические тесты должны пройти со статусом «успешно».

5.2. Запуск автоматических тестов CI

Полный прогон в CI выполняется автоматически на каждый pull request и push в main через workflow `.github/workflows/ci.yml`. На каждый push в main также запуска-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.09.03-04 51 01-1

ется workflow `.github/workflows/coverage-badge.yml`, обновляющий SVG-бейдж покрытия в `.github/badges/jacoco.svg`. Прогресс прогона можно отслеживать в разделе Actions репозитория GitHub. Локально полный прогон осуществляется командой `make test` (если Makefile предоставлен) или последовательностью команд из таблицы выше.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

6.1. Испытание создания и редактирования workflow

Целью данной методики является проверка корректности работы визуального редактора workflow.

Порядок проведения испытания:

- 1) Открыть приложение по адресу <http://localhost:4200>. Проверить наличие списка workflow и кнопки «Создать workflow».
 - 2) Нажать «Создать workflow», ввести имя и описание, подтвердить. Проверить появление пустого холста с палитрой узлов слева.
 - 3) Перетащить узел из палитры (например, «HTTP») на холст через drag-and-drop. Проверить появление узла на холсте, статусную индикацию (серый — pending).
 - 4) Перетащить ещё один узел (например, «Filter»). На выходном порту HTTP-узла зажать левую кнопку мыши и протянуть до входного порта Filter-узла. Проверить создание связи (SVG-кривая Безье с указателем направления).
 - 5) Кликнуть на HTTP-узел. Проверить открытие панели инспектора справа с полями URL, метод, заголовки, тело, тайм-аут.
 - 6) Заполнить поля: URL = <https://httpbin.org/get>, метод = GET, тайм-аут = 30. Нажать «Сохранить» в инспекторе.
 - 7) Кликнуть на Filter-узел, ввести выражение фильтрации (например, `item.id > 0`), нажать «Сохранить».
 - 8) Нажать общую кнопку «Сохранить» workflow. Проверить появление сообщения «Workflow saved».
 - 9) Перезагрузить страницу (F5). Открыть тот же workflow. Проверить восстановление: оба узла на холсте, конфигурация в инспекторе сохранена, связь между ними отображается.
 - 10) Дополнительные действия: zoom через колесо мыши, pan через тяну-и-брось пустого пространства, удаление узла через клавишу Delete, копирование через Ctrl+C / Ctrl+V.
- Критерий успешного прохождения: все шаги выполняются без ошибок, конфигурация сохраняется, при перезагрузке восстанавливается.

6.2. Испытание системы триггеров

Целью данной методики является проверка корректности работы трёх типов триггеров.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Ручной триггер. Создать workflow с одним HTTP-узлом на <https://httpbin.org/get>. Нажать кнопку «Запустить». Проверить: появление желтого узла (running), переход в зелёный (success) через секунды; в панели «История запусков» — новая запись со статусом success, длительностью.

Scheduler-триггер. Открыть панель «Триггеры». Нажать «Добавить триггер», выбрать тип «Scheduler», ввести cron `*/1 * * * *`, включить enabled, сохранить. Подождать 2–3 минуты. Проверить: в панели «История запусков» появляются записи с интервалом 1 минута, `trigger_type = scheduler`, статус success.

Webhook-триггер. Добавить триггер типа «Webhook», скопировать сгенерированный URL. Через curl: `curl -X POST http://localhost:8080/v1/webhook/{token} -H "Content-Type: application/json" -d '{"hello":"world"}'`. Проверить ответ 202 Accepted; в панели «История запусков» — новая запись, `trigger_type = webhook`, в детализации первого узла — тело `{"hello":"world"}` как вход.

Критерий успешного прохождения: все три типа триггеров инициируют запуск, входные данные корректно проходят в первый узел workflow.

6.3. Испытание HTTP-узла

Целью данной методики является проверка исполнения HTTP-узла для разных методов и сценариев.

Порядок проведения испытания:

- 1) workflow HTTP GET <https://httpbin.org/get?param=value> — ожидается success, в output эхо параметра;
- 2) workflow HTTP POST <https://httpbin.org/post> с заголовком Authorization и JSON-телом — ожидается success, эхо в output;
- 3) workflow HTTP GET с невалидным URL — ожидается failed, error_message о DNS/connection;
- 4) workflow HTTP GET с тайм-аутом 3 секунды на <https://httpstat.us/200?sleep=10000> — ожидается failed по тайм-ауту;
- 5) workflow HTTP DELETE <https://httpbin.org/delete> — ожидается success.

Критерий успешного прохождения: все методы исполняются корректно, ошибки диагностируются с осмысленными сообщениями.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6.4. Испытание потоков данных

Целью данной методики является проверка пять операций потоков данных.

Порядок проведения испытания:

- 1) workflow HTTP → Filter с выражением `item.age > 18` — проверить фильтрацию объектов с возрастом ≤ 18 из выхода HTTP;
- 2) workflow HTTP → Map с выражением `{name: item.name.toUpperCase()}` — проверить преобразование имени в верхний регистр;
- 3) workflow HTTP → Reduce с выражением `acc + item.price`, начальным значением 0 — проверить суммирование цен;
- 4) workflow HTTP → Foreach (логирование) — проверить, что `foreach` возвращает входной список без изменений и регистрирует `side-effect`;
- 5) workflow HTTP → Flatmap с выражением `item.children` — проверить разворачивание вложенных списков `children` в плоский список.

Критерий успешного прохождения: каждая операция возвращает ожидаемый результат, тип данных корректно сериализуется.

6.5. Испытание Code Node

Целью данной методики является проверка корректное исполнение и полную изоляцию пользовательского Python- и JavaScript-кода.

Шаги (Python):

- 1) Корректный код `print(json.dumps({"sum": sum(json.load(sys.stdin)["numbers"])}))` с входом `{"numbers": [1,2,3]}` — ожидается success, output `{"sum": 6}`;
- 2) `urllib.request.urlopen(...)` — ожидается failed, «Sandbox network denied»;
- 3) `open("/etc/test", "w")` — ожидается failed, «Read-only file system»;
- 4) `x=[0]*(10**8)` — ожидается failed, OOM;
- 5) `while True: pass` — ожидается failed, «Sandbox timeout» через 5 секунд;
- 6) `import subprocess; subprocess.run(["sudo", "id"])` — ожидается failed, privilege escalation заблокирован.

Шаги (JavaScript). Аналогичные шесть сценариев выполняются для узла Code Node на языке JavaScript с базовым образом `node:20-alpine`: корректный код суммирования чисел через `process.stdin`; попытка `require('http').get(...)`; попытка `fs.writeFileSync('/etc/test', ...)`; попытка выделения большого массива `new Array(1e8).fill(0)`; бесконечный цикл

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

while (true) {}); попытка child_process.execSync('sudo id'). Ожидаемые результаты совпадают с Python: первый сценарий завершается успехом, шесть атакующих — со статусом failed и осмысленными диагностическими сообщениями.

Критерий успешного прохождения: корректный код на обоих языках работает; все шесть атакующих сценариев блокируются как в Python-, так и в JavaScript-вариантах.

6.6. Испытание параллельного исполнения и отказоустойчивости

Целью данной методики является проверка параллельное исполнение и изоляцию сбоев.

Порядок проведения испытания:

- 1) workflow с двумя параллельными ветвями: A (Code Node Python time.sleep(3)) + B (Code Node JS setTimeout(3000)); A → C; B → D — ожидается: A и B становятся жёлтыми одновременно, через 3 с — зелёными, общее время ≈ 6 с (не 12 с);
- 2) workflow: A (HTTP с невалидным URL) + B (корректный Code Node); A → C; B → D — ожидается A red, C grey (skipped «Dependency failed»), B green, D green, workflow failed;
- 3) Цепочка A → B → C → D, где B сбоит — ожидается A green, B red, C grey, D grey (транзитивный skip);
- 4) workflow с циклом A → B → A — ожидается failed «Cycle detected» сразу без запуска узлов.

Критерий успешного прохождения: параллелизм работает, ошибки изолируются, циклы детектируются.

6.7. Испытание интерфейса просмотра состояния и мониторинга

Целью данной методики является WebSocket-трансляция событий, визуальная индикация в редакторе, история запусков.

Порядок проведения испытания:

- 1) В редакторе запустить workflow из двух последовательных узлов. Наблюдать переход подсветки с первого узла на второй.
- 2) Запустить workflow с сбойным узлом. Проверить красную подсветку и сообщение об ошибке в инспекторе.
- 3) Открыть панель «История запусков», выбрать любой исторический запуск. Проверить: список всех node_run с статусами, кликом по узлу — входные/выходные данные и сообщения об ошибках.
- 4) Проверить через DevTools браузера (Network → WS) поступление STOMP-сообщений от сервера при запуске workflow.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Критерий успешного прохождения: события транслируются в реальном времени, история сохраняется корректно.

6.8. Испытание версионирования

Целью данной методики является создание ревизий и именованных версий, корректность отката.

Порядок проведения испытания:

- 1) Создать workflow, сохранить (revision 1). Изменить и сохранить дважды (revision 2, 3).
- 2) Открыть панель «Версии». Проверить наличие трёх ревизий с временами создания.
- 3) Создать именованную версию: нажать «Создать snapshot», ввести тег v1.0, сохранить. Проверить появление в списке.
- 4) Изменить ещё раз, сохранить (revision 4). Восстановить версию v1.0. Проверить: на холсте отобразилась конфигурация revision 1; в списке появилась revision 5 с тем же графом, что у revision 1.
- 5) Прямой SQL: `SELECT revision_number, graph_skeleton FROM workflow_revision WHERE workflow_id = '<id>' ORDER BY revision_number;` — 5 записей, revisions 2–4 неизменны (иммутабельность).

Критерий успешного прохождения: версии создаются, откат работает, иммутабельность исторических данных подтверждена.

6.9. Испытание content-addressed-хранения в MinIO

Целью данной методики является дедупликация идентичных конфигов узлов.

Порядок проведения испытания:

- 1) Создать workflow A с HTTP-узлом, URL `https://example.com`. Сохранить.
- 2) Создать workflow B с идентичным HTTP-узлом. Сохранить.
- 3) SQL: `SELECT content_hash, COUNT(*) FROM blob_index;` — для соответствующего хеша одна запись.
- 4) Открыть MinIO Console `http://localhost:6006`, bucket `alla-blobs`, директория `blobs/` — нет дублирующих файлов.
- 5) Изменить конфиг в workflow A. Проверить: добавилась одна запись в `blob_index` (новый хеш), запись для исходного осталась.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Критерий успешного прохождения: одинаковые конфиги хранятся однократно; разные — раздельно.

6.10. Испытание автодокументации REST API

Целью данной методики является корректность работы Swagger UI.

Порядок проведения испытания:

- 1) Открыть <http://localhost:8080/v1/swagger-ui.html>. Проверить заголовок «FluxPilot API», группировку эндпойнтов по тегам.
- 2) Раскрыть POST `/v1/workflows/{id}/runs`. Проверить описание, схемы request/response, примеры.
- 3) «Try it out» → Execute с валидным id — получить корректный ответ.
- 4) Открыть <http://localhost:8080/v1/v3/api-docs> — валидная JSON-спецификация OpenAPI 3.0.

Критерий успешного прохождения: документация полная, актуальная и валидная.

6.11. Испытание адаптивной вёрстки клиентской части

Целью данной методики является корректная работа на разных разрешениях.

Порядок проведения испытания: через DevTools браузера (responsive mode) последовательно установить разрешения 480×800, 640×960, 900×1280, 1200×1600, 1920×1080. Для каждого:

- 1) Открыть список workflow, проверить читаемость и доступность всех элементов;
- 2) Открыть редактор, проверить наличие холста, палитры, инспектора;
- 3) Открыть панель истории запусков.

Критерий успешного прохождения: интерфейс остаётся функциональным и читаемым на всех пяти разрешениях; на узких экранах элементы перегруппируются (палитра становится выпадающим меню, инспектор — модальной панелью).

6.12. Испытание надёжности

Целью данной методики является устойчивость к некорректным входным данным.

Порядок проведения испытания:

- 1) Создать workflow с пустым URL HTTP-узла, попытаться сохранить — ожидается ошибка валидации в инспекторе, сохранение блокируется;
- 2) Ввести в cron-выражение невалидную строку (hello), сохранить триггер — ожидается ошибка валидации;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3) Запустить workflow с заведомо сбойным узлом, дождаться завершения — проверить: данные запуска и предыдущих успешных узлов сохранены в БД (никакой rollback всего workflow при сбое одного узла);

4) Принудительно остановить backend во время исполнения workflow (через docker stop контейнера), запустить заново — проверить: незавершённый workflow_run остался в статусе running, после следующего штатного запуска backend помечается как interrupted или восстанавливается (зависит от реализации recovery-логики).

Критерий успешного прохождения: приложение не падает; ошибки логируются; данные не теряются.

6.13. Испытание режима пошаговой отладки workflow

Целью данной методики является проверка корректности работы режима пошаговой отладки — запуска одной выбранной ноды workflow с произвольным входным значением без полного прогона графа.

Порядок проведения испытания:

- 1) Создать workflow, состоящий из нескольких связанных узлов (например, HTTP-узел → Filter-узел → Code Node), сохранить.
- 2) В интерфейсе редактора выбрать произвольную ноду графа (например, Filter-узел в середине цепочки) и инициировать отладочный запуск с произвольным JSON-телом в качестве входа.
- 3) Проверить, что результат исполнения возвращается клиентской части в синхронном виде и отображается в инспекторе во вкладках входа и выхода.
- 4) Проверить, что предшествующие узлы графа не исполнялись (отсутствуют записи о них среди событий запуска).
- 5) Открыть панель «История запусков»: проверить наличие записи об отладочном запуске с пометкой отладочного исполнения и возможностью просмотра деталей.
- 6) Повторить шаги 2–5 с заведомо некорректным входом, ожидаемо приводящим к ошибке исполнения (например, нарушающим типизацию выражения фильтрации): проверить, что диагностическое сообщение об ошибке корректно возвращается клиентской части и фиксируется в записи запуска.

Критерий успешного прохождения: отладочный запуск исполняет только выбранную ноду, возвращает результат в синхронном виде, корректно обрабатывает ошибки и фиксирует запуск в общей истории запусков с пометкой отладочного исполнения.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

7. ИСПЫТАНИЕ НЕФУНКЦИОНАЛЬНЫХ ХАРАКТЕРИСТИК

7.1. Производительность REST API

Для каждого из пяти ключевых эндпойнтов выполнить 50 последовательных запросов через curl с замером `time_total`, рассчитать min, avg, 95-й перцентиль. Эндпойнты: GET /v1/workflows, GET /v1/workflows/{id}, GET /v1/workflows/{id}/runs, GET /v1/workflow-runs/{runId}, GET /v1/swagger-ui.html. Критерий: 95-й перцентиль не превышает 500 мс.

7.2. WebSocket-задержка

Подключиться к STOMP-топику, замерить интервал между генерацией события на сервере (логированием в backend) и получением сообщения клиентом (timestamp в DevTools). Серию из 20 замеров. Критерий: 95-й перцентиль не превышает 200 мс в локальной сети.

7.3. Покрытие тестами

После `mvn verify` зафиксировать процентные показатели из `backend/target/site/jacoco/index.html` (строки, ветви). Критерий: общее покрытие не ниже 90 %.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8. СВОДНЫЕ ПОКАЗАТЕЛИ

Показатель	Значение
Серверные модульные тесты (JUnit 5)	около 55
Серверные интеграционные тесты (Testcontainers)	около 12
Клиентские модульные тесты (Jasmine + Karma)	около 65
End-to-end тесты (Playwright)	около 95
Всего автоматических тестов	около 230
Доля успешных тестов	100 %
Покрытие серверной части по JaCoCo (строки)	не менее 90 %
95-й перцентиль REST API на чтение	не более 500 мс
95-й перцентиль WebSocket в локальной сети	не более 200 мс
Время от триггера до первого узла running	не более 5 с
Время отклика веб-интерфейса	не более 2 с
Время полного прогона CI	не более 12 минут
Готовность системы после docker compose up -d	не более 60 с
Адаптивная вёрстка — корректные брейкпойнты	480, 640, 900, 1200, 1920 пикселей

Таблица 2. Сводные целевые показатели команды к моменту защиты

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

9. ПРИЛОЖЕНИЕ. ТРАССИРОВКА ТРЕБОВАНИЙ ОБЩЕГО ТЗ К РАЗДЕЛАМ ПМИ

Требование общего ТЗ	Раздел ПМИ	Краткий метод
(1) Визуальный редактор workflow	6.1	Ручной (10 шагов)
(2) Система триггеров	6.2	Ручной (3 типа)
(3) HTTP-узел	6.3	Ручной (5 сценариев)
(4) Узлы потоков данных	6.4	Ручной + автоматический (DataflowNodeExecutorsTest)
(5) Code Node + sandbox	6.5	Ручной (12 сценариев: 6 Python + 6 JS)
(6) Параллельное исполнение	6.6	Ручной (4 сценария) + интеграционный
(7) Версионирование	6.8	Ручной + прямые SQL-запросы
(8) Просмотр состояния	6.7	Ручной + DevTools WebSocket
(9) Feature-флаги	индивидуальная ПМИ Клушина А.А. (RU.17701729.09.03-04 51 03-1, раздел 5.9)	Покрывается узлом BranchSplit в режиме «Feature Flag» (стратегии attribute и percentage) — см. также (12)
(10) WebSocket-трансляция	6.7	Ручной через STOMP-клиент
(11) MinIO content-addressed	6.9	Ручной + MinIO Console + SQL
(12) A/B-аналитика workflow	индивидуальная ПМИ Клушина А.А. (RU.17701729.09.03-04 51 03-1, раздел 5.9)	Модульные и интеграционные тесты узлов BranchSplit/BranchMerge и серверного модуля аналитики; ручная проверка REST-эндпойнта /v1/workflows/{id}/ab-analytics
(13) Снапшоты workflow	6.8	Ручной через REST API + панель «Версии»
(14) Режим пошаговой отладки workflow	6.13	Ручной через интерфейс редактора + прямой SQL-запрос

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Требование общего ТЗ	Раздел ПМИ	Краткий метод
		к таблицам workflow_run и node_run

Таблица 3. Трассировка требований ТЗ-4.1.1 к методикам испытаний

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. JUnit 5 User Guide [Электронный ресурс]. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 08.04.2026).
2. Testcontainers for Java [Электронный ресурс]. — URL: <https://java.testcontainers.org/> (дата обращения: 12.04.2026).
3. JaCoCo Code Coverage Library [Электронный ресурс]. — URL: <https://www.jacoco.org/jacoco/trunk/doc/> (дата обращения: 15.04.2026).
4. Playwright Documentation [Электронный ресурс] / Microsoft. — URL: <https://playwright.dev/> (дата обращения: 18.04.2026).
5. Jasmine Documentation [Электронный ресурс]. — URL: <https://jasmine.github.io/> (дата обращения: 20.04.2026).
6. Karma Test Runner [Электронный ресурс]. — URL: <https://karma-runner.github.io/> (дата обращения: 22.04.2026).
7. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]